

# Document Intelligence Platform

Intelligent Document Extraction, Validation & API Platform

# Problem & Objective

## The problem

Financial teams receive invoices & statements as native/scanned PDF, JPG or PNG in inconsistent layouts — manual extraction is slow and error-prone.

## The objective

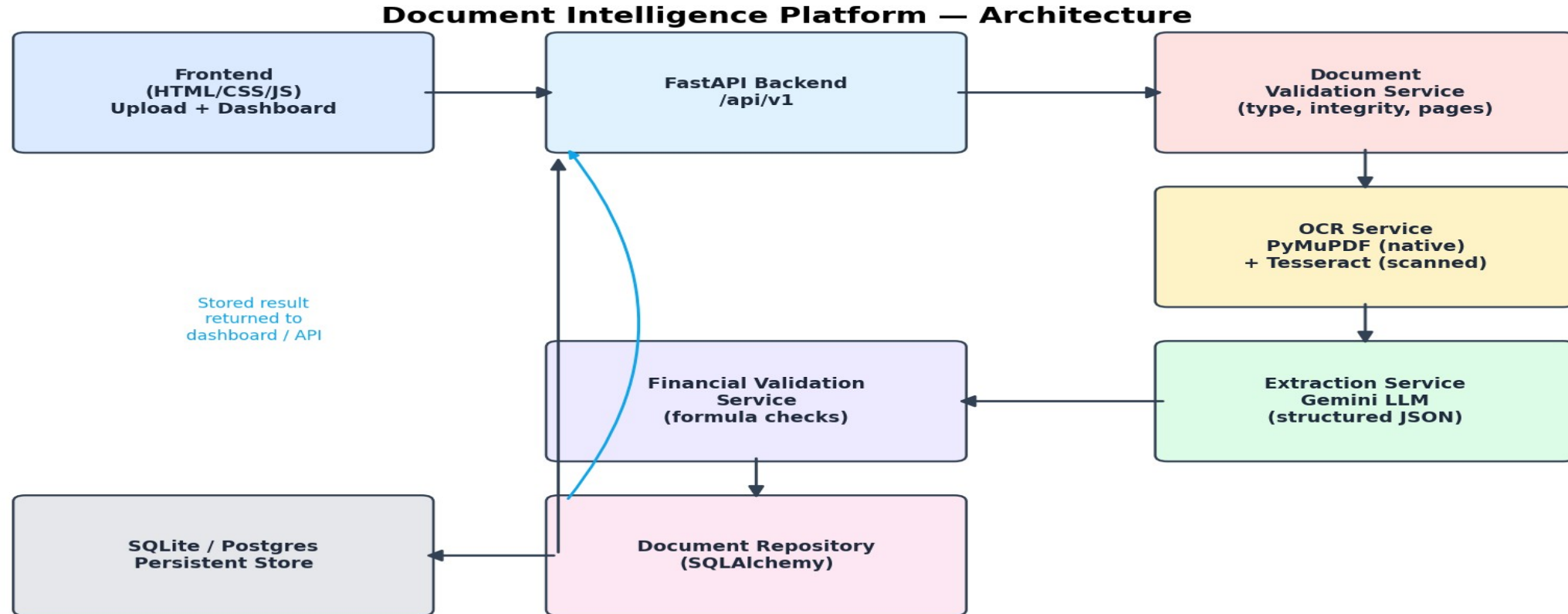
Build & deploy a reusable Document Intelligence service that extracts all available fields, validates key financial relationships, and exposes results via a dashboard and REST API.

## Documents in scope

- Invoices
- Balance Sheets
- Profit & Loss Statements
- Cash Flow Statements

# Architecture

Upload → Validate → OCR → AI Extraction → Financial Validation → Store → Dashboard/API



Errors at any stage (validation/OCR/LLM/DB) are caught and returned as a controlled FAILED JSON response instead of a stack trace.

# Technology Stack

| Concern         | Choice                          | Why                                                           |
|-----------------|---------------------------------|---------------------------------------------------------------|
| API framework   | FastAPI                         | Async, auto Swagger/OpenAPI docs, Pydantic validation         |
| Frontend        | HTML/CSS + vanilla JS (Jinja2)  | Single deployable service, meets brief's frontend requirement |
| Native PDF text | PyMuPDF                         | Fast, accurate embedded-text extraction                       |
| OCR             | Tesseract (pytesseract)         | Free, offline, no API key needed                              |
| AI extraction   | Google Gemini (structured JSON) | Generous free tier, native JSON mode                          |
| Database        | SQLite (Postgres-ready)         | Zero-setup persistence, swappable via DATABASE_URL            |
| Deployment      | Docker on Render                | One image; free tier; Tesseract baked in                      |

# REST API Design

Swagger/OpenAPI docs available at </docs> on the deployed environment

**POST**

**`/api/v1/documents/process`**

Upload & process a PDF/JPG/PNG

**GET**

**`/api/v1/documents/{document_name}`**

Latest structured result by document name

**GET**

**`/api/v1/documents`**

List processed documents (dashboard feed)

**GET**

**`/api/v1/health`**

Health check

# Extraction & Financial Validation

## Extraction

- ALL meaningful fields extracted, not a fixed list
- Missing values → null, never invented
- Evidence: source\_text + page\_number per field

## Validation

- Formula, inputs, calculated vs reported value, variance
- PASS / FAIL / NOT\_APPLICABLE — never assumes missing fields
- Tolerance: max(abs 1.0, 1% of reported value)

```
"invoice_total_check"  
formula: subtotal + tax - discount  
calculated_value: 13125.00  
reported_value: 13125.00  
variance: 0.00  
status: PASS
```

# Testing & Deliverables

## Testing

- All 4 document types processed end-to-end
- Scanned/image-based document included
- Unsupported file & missing-field scenarios included
- pytest: file validation, financial calc, API flow (14 tests, all passing)

## Deliverables

- Public GitHub repo (backend, frontend, tests, docs)
- Deployed frontend + API + Swagger URLs
- README (setup, architecture, API, limitations)
- Architecture diagram + sample JSON outputs

# Known Limitations & Production Roadmap

## Known limitations

- Multi-period statements validated as a single period, not per-year
- Document type is user-selected, not auto-classified (per brief)
- OCR quality varies on low-resolution/skewed scans

## For production

- Async processing queue instead of synchronous requests
- Per-period structured extraction & validation
- Managed Postgres + object storage + auth/rate-limiting